

A5. Diseño del producto

El equilibrio entre la estética (**UI**) y la utilidad/eficiencia (**UX**) es exactamente el punto donde la **ingeniería de software se encuentra con el diseño de producto**. Una aplicación con una interfaz espectacular que tarda 5 segundos en responder o que se cierra inesperadamente por falta de memoria es descartada de inmediato; por otro lado, una app técnicamente perfecta pero visualmente tosca genera desconfianza y fricción.

Para lograr este punto de equilibrio sin comprometer el rendimiento técnico ni la experiencia de uso, se aplican los siguientes lineamientos, patrones de diseño y recomendaciones de arquitectura gráfica.

A.5.1 Regla de Oro: La forma sigue a la función (*Form Follows Function*)

En el diseño de software móvil, una decisión visual nunca debe ser puramente decorativa si afecta la usabilidad o la velocidad de renderizado.

Lineamientos de Equilibrio

- **Retroalimentación visual inmediata (Límite de los 100 ms):**
Cualquier acción del usuario (un tap, un *swipe*) debe responder visualmente o con *haptic feedback* en menos de 100 ms. Si la operación de backend o base de datos tarda más, la interfaz **debe cambiar inmediatamente a un estado de carga** o aplicar una **Interfaz Optimista (*Optimistic UI*)**.
- **Jerarquía visual dirigida por intención:**
No todo en la pantalla debe competir por la atención del usuario. Aplica contraste de tamaño y peso tipográfico para guiar el ojo hacia la **Acción Principal (*Primary Call to Action - CTA*)**. Una pantalla debe tener **un solo botón primario**; las acciones secundarias deben ser visualmente discretas.
- **Respeto a las convenciones de plataforma (Patrón del usuario cautivo):**
No reinventes patrones de interacción consolidados. Los usuarios de iOS esperan deslizamientos laterales para regresar (*Swipe to Back*) y barras de pestañas inferiores (*TabBars*); los de Android están acostumbrados a patrones de navegación Material. Romper estas convenciones por buscar "originalidad visual" incrementa la carga cognitiva.

A.5.2 Patrones de diseño visual (UI/UX) orientados a rendimiento y estética

A continuación, se resumen los patrones clave que logran que una aplicación se vea moderna y elegante mientras optimizan los recursos del sistema:

Patrones de carga y transición de estado

Patrón	Solución UX / UI	Beneficio Técnico y de Performance
Skeleton Screens (Pantallas Esqueleto)	Sustituye los indicadores circulares (<i>spinners</i>) por siluetas grises animadas que imitan la forma del contenido final.	Reduce la latencia percibida (<i>Perceived Latency</i>). El usuario siente que la app es instantánea porque la estructura visual ya está presente mientras se descarga el payload JSON.
Progressive Image Loading (Carga Progresiva)	Muestra una versión mínima en baja resolución blurred o un <i>placeholder</i> vectorial mientras la imagen en HD se descarga de la CDN.	Evita saltos de maquetado (<i>Layout Shifts</i>) en el árbol de UI y reduce el consumo inicial de memoria RAM al no decodificar mapas de bits masivos de golpe.
Optimistic UI (Interfaz Optimista)	Refleja el cambio visual de inmediato (ej. marcar un favorito) asumiendo éxito en la API, revirtiendo suavemente si hay error.	Elimina la percepción de retardo por latencia de red, haciendo que la aplicación se sienta nativa y ultrafluida.

Patrones de estructura visual y layout

- **Modal bottom sheets en lugar de diálogos centrales:** Los diálogos emergentes en el centro de la pantalla bloquean el contexto y son incómodos de cerrar con una sola mano. Las hojas inferiores (*Bottom Sheets*) se deslizan desde la parte inferior, son naturales para el pulgar y permiten gestos intuitivos para descartar (*Drag down to dismiss*).
- **Card-based layouts (Estructura en tarjetas):** Agrupar la información en contenedores con bordes redondeados y una suave sombra o borde de 1 dp permite separar visualmente bloques de datos complejos. Técnicamente facilita la reutilización de

componentes inmutables en listas infinitas (`ListView.builder` / `LazyColumn`).

- **Design tokens para consistencia:** Usa un sistema centralizado de variables (*Tokens*) para gestionar colores, radios de borde (12dp, 16dp) y espaciados basados en la regla de los 8pt (8dp, 16dp, 24dp, 32dp). Esto evita incoherencias estéticas y acelera el desarrollo en código.

A.5.3 Recomendaciones prácticas para no romper la funcionalidad por "Ver la App Bonita"

1. **Cuidado con las animaciones excesivas:** Las animaciones agregan dinamismo y elegancia, pero las transiciones que duran más de 300 ms aburren al usuario frecuente y consumen recursos de GPU. Mantén las animaciones de transición entre 150 ms y 250 ms usando curvas de aceleración naturales (*Ease-out* / *Spring Physics*).
2. **Evita el abuso de efectos de gran carga gráfica (*Blur* / *Glassmorphism*):** Los efectos de cristal translúcido (*Backdrop Filter* / *Blur*) requieren un procesamiento intensivo de procesamiento gráfico por cuadro. Si se usan indiscriminadamente dentro de una lista con desplazamiento (*Scroll*), provocarán **pérdida de cuadros (*Jank*)** en dispositivos de gama media o baja.
3. **Diseña exhaustivamente los estados mínimos de la UI:** Una pantalla "bonita" no es solo la que tiene datos de prueba perfectos. Debes diseñar obligatoriamente los 5 estados de cada vista:
 - **Ideal:** Con datos completos.
 - **Empty:** Sin datos registrados aún (con ilustración amigable e instrucción de qué hacer).
 - **Loading:** Mientras conecta a la red.
 - **Error:** Cuando falla el servidor o la red (ofreciendo un botón de "Reintentar").
 - **Partial:** Cuando la información está incompleta o limitada.